

Assignment 1 – Entity-Component Framework [Part 2] -Chess-Game

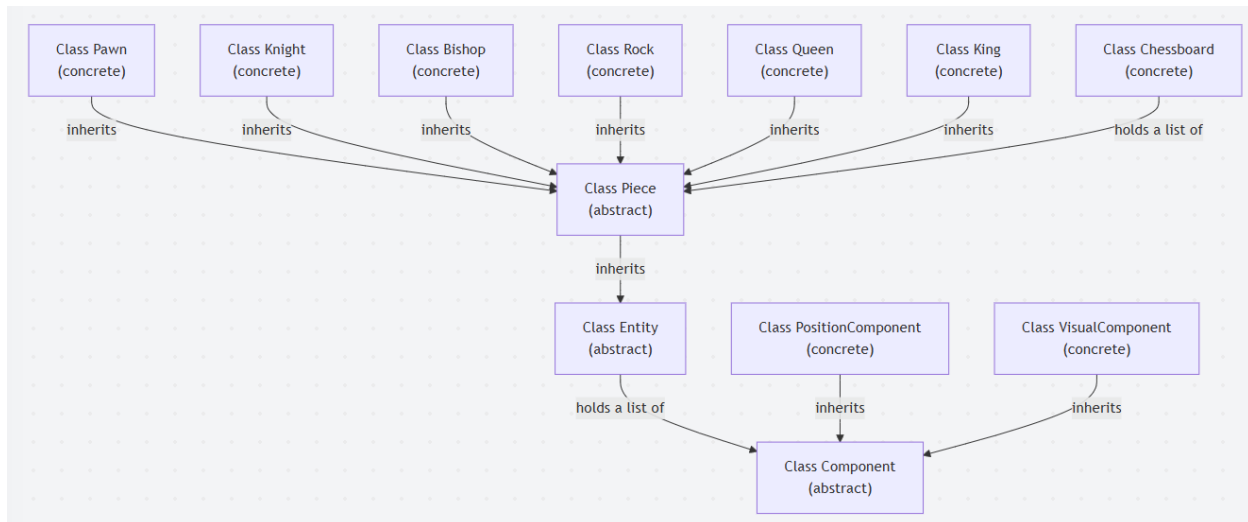
Goal

To put into practice the object-oriented programming principles and techniques covered in class, including inheritance, polymorphism and runtime type identification (static casting and dynamic casting). The goal of this assignment is to extend the Entity-Component framework for game development to implement a chess game. It is an exercise in which students should implement a 2 player chess game using the framework.

The resulting game should be a 2 (human) player game in which both participants take turns to input their chess moves via keyboard commands. The game should be a text-based terminal application that reads player input from the stdin and displays the information and board layout to the stdout. So a command like a3 d5 should be read as: take the piece at location A3 and move it to position D5. Player 1 will be controlling the white pieces and player 2 will control the black ones. As per-chess rules, the white player will go first.

Description

To develop this assignment, you must use the framework and extend the classes using polymorphism as they are presented in the following figure:



The *Entity* class, *Component* class, *PositionComponent* class and *VisualComponent* class are provided in the *framework.hh* and *framework.cc* and you do not need to implement them. The *PositionComponent* class stores the position of the entity attached to it and the *VisualComponent* presents visually the entity attached to it.

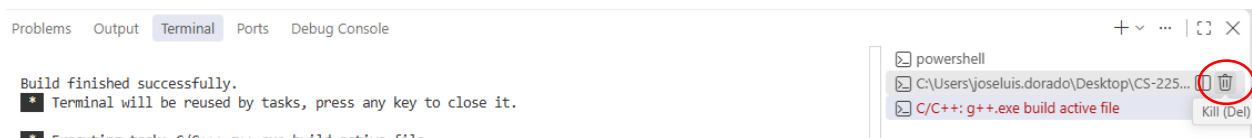
The *Piece* class and the *Chessboard* class are provided partially. You must implement the methods indicated in the header file **chess_base.hh**. The *Piece* class is an abstract class that manage some common behavior of all chess pieces, and Chessboard class manages the board, which is represented as a 2D array of Pieces objects, where an empty square in the board is stored as a null pointer. You have to implement the methods in *Chessboard* class and the *Piece* class in the **chess_base.cc** file. You should NOT make changes in header file **chess_base.hh**.

The *Piece* class have some pure virtual functions that must be implemented in the classes deriving from him. You have to extend the Piece class for all the 6 different pieces of the chess and override the corresponding methods. The classes deriving from Piece must be implemented in the **chess_pieces.hh** and the **chess_pieces.cc** files. The virtual method **canMoveTo(int x, int y)** must validate if the movement of the specific Piece is valid according to the rules of chess.

You do NOT have to implement the chess game logic and draw the board dynamically, as this is actually implemented in the **main.cpp**. This exercise is focused on applying the concepts of polymorphism and runtime type identification. It is mandatory to use the at least **dynamic_cast** or **static_cast** to perform upcasting and downcasting in this assignment.

Note

In Visual Studio Code after running the **main.cpp** successfully, you could get an error **Permission denied collect2.exe: error: ld returned 1 exit status**. In that case, you must kill the terminal by pressing the trash icon at the right top bar. This error happens because Visual Studio Code does not kill the process running the program automatically and must be terminated manually.



What to do:

The code explicitly requires the following files, which the student should create and submit:

- chess_base.cc, chess_pieces.hh and chess_pieces.cc

Submission Guidelines

Submit only chess_base.cc, chess_pieces.hh, chess_pieces.cc as a zip folder on Moodle with the following naming conventions:

cs225fa26_{username}_1.2.zip